



Campus Nacional

Momento de Retroalimentación: Reto Selección, configuración y entrenamiento del modelo

Hector Kenneth Ramos Velazquez | A01642284

Elisheba hannai trejo leyva | A01736967

Fecha: 11 de septiembre del 2026.

En esta fase del proyecto, se realizó la configuración, la comparación y el entrenamiento de varios modelos de aprendizaje automático con el conjunto de datos REHAB, que se centra en el análisis de movimientos ejecutados en actividades rehabilitadoras. El propósito principal fue encontrar un modelo que pudiera clasificar correctamente las distintas actividades basándose en los atributos obtenidos de los datos de los sensores.

Se tomó como base el procesamiento que se llevó a cabo en fases previas del proyecto. Se emplearon 24 características por cada observación, las cuales se derivaron de seis canales del Sensor 2 a través de cuatro mediciones estadísticas: media, desviación estándar, mínimo y máximo. Además, se continuó excluyendo la actividad 014, a causa de que tiene menos observaciones que las demás clases.

Con el objetivo de disminuir potenciales fuentes de sesgo o fuga de información, se llevó a cabo una evaluación adicional de la calidad de los datos antes del entrenamiento. Se llevaron a cabo la eliminación de observaciones totalmente nulas y registros duplicados antes de dividir el conjunto de datos. Por lo tanto, se consiguió un grupo final de 3.620 observaciones que pertenecen a 15 clases. Este conjunto fue dividido de forma estratificada: el 70% se utilizó para el entrenamiento, el 15% para la validación y el otro 15% para la prueba.

Se entrenaron y evaluaron cuatro algoritmos de clasificación basados en esta división común: Árbol de decisión, K-Nearest Neighbors (KNN), Support Vector Machine (SVM) y bosque aleatorio. Se evaluaron diferentes configuraciones de hiperparámetros para cada modelo, empleando el conjunto de validación como criterio de selección y dejando el conjunto de prueba para la evaluación final. Se examinó el desempeño de los modelos utilizando métricas como la precisión, el recall, la exactitud y el F1-score, prestando particular atención al macro F1-score para evaluar equitativamente el rendimiento en las diferentes clases.

En última instancia, los resultados logrados hicieron posible hacer una comparación objetiva de las opciones evaluadas y elegir la configuración con mayor capacidad de generalización para el problema de clasificación que se propuso.

Comparación de modelos para REHAB

1. Tipo de modelo requerido

El problema requiere un modelo de clasificación multiclase supervisada, por que se busca predecir la actividad de rehabilitación correspondiente a cada muestra a partir de características numéricas y etiquetas conocidas.

la variable objetivo es activity, con 15 clases: 000 a 013 y 015

2. Tipos de datos y modelos compatibles

Se utilizó el dataset compartido de REHAB, que representa las señales temporales mediante datos tabulares. Cada muestra contiene 24 características: media,desviación estándar, mínimo y máximo de los canales f1,f2,f3,f4,f5 y pitch3.

Se conservó la exclusión de la actividad 014, siguiendo la decisión del análisis exploratorio y la recomendación del profesor documentadas en el proyecto inicial.

Conjunto	Muestras	Características	Clases
Entrenamiento	2,534	24	15
Validación	543	24	15
Prueba	543	24	15
Total	3,620	24	15

Se verificó que los archivos no presentan valores faltantes, infinitos, filas duplicadas dentro de cada conjunto ni vectores de características exactamente iguales compartidos entre conjuntos.

KNN, Decision Tree, Random Forest y SVM son compatibles con estas entradas numéricas y con la clasificación de múltiples actividades

3. Investigación de modelos y elección del más adecuado

Se investigaron cuatro alternativas utilizando la documentación oficial de scikit-learn:

Modelo	Funcionamiento y justificación
KNN	Clasifica utilizando las etiquetas de los vecinos más cercanos. Permite evaluar la similitud entre las características de las actividades. Depende de la escala, la distancia y el número de vecinos
Decision Tree	Aprende reglas basadas en umbrales de las variables. Representa relaciones no lineales y permite controlar su complejidad mediante la profundidad y el tamaño mínimo de las hojas. Puede sobreajustar si el árbol es demasiado complejo
Random Forest	Combina múltiples árboles entrenados con aleatoriedad en muestras y características. Permite comparar un conjunto de árboles con un árbol individual y puede reducir su variabilidad
SVM	Aprende fronteras de separación entre clases. Los kernels lineal y RBF permiten comparar fronteras lineales y no lineales, con diferentes niveles de regularización

Random Forest fue seleccionado como el modelo más adecuado entre las configuraciones probadas, por que obtuvo el mayor F1 macro de validación: 90.42%. Esta selección se realizó antes de consultar los resultados de la prueba.

4. Configuración, entrenamiento y desempeño

Se utilizaron las implementaciones de scikit-learn. Todos los modelos recibieron los mismos archivos de entrenamiento, validación y prueba.

KNN y SVM incorporaron StandardScaler dentro de un pipeline. La estandarización se ajustó exclusivamente con entrenamiento y se aplicó sin reajuste a validación y prueba. Decision tree y Random Forest utilizaron las variables sin estandarizar.

Modelo	Parámetros probados	Combinaciones
Decision tree	max_depth: 3, 5, 10, None; min_samples_leaf: 1, 2, 5, 10	16
Random Forest	n_estimators: 100, 200; max_depth: 5, 10, None; min_samples_leaf: 1, 5	12
KNN	n_neighbors: 1, 3, 5, 7, 9; weights: uniform, distance; metric: euclidean, manhattan	20
SVM	Kernel lineal con C: 0.1, 1, 10; kernel RBF con C: 0.1, 1, 10 y gamma: scale, auto	9

None indica que no se establece un límite explícito de profundidad. Decision Tree y Random Forest utilizaron random_state=42. Random Forest también utilizó max_features="sqrt" y n_jobs=-1.

Configuraciones seleccionadas en validación

Modelo	Mejor Configuración
Decision Tree	max_depth=None, min_samples_leaf=1
Random Forest	n_estimators=100, max_depth=None, min_samples_leaf=1, max_features="sqrt"

KNN	n_neighbors=1, weights="uniform", metric="manhattan"
SVM	kernel="rbf", C=10, gamma="scale"

Resultados de Validación

Todas las métricas se expresan en porcentaje.

Modelo	Accuracy	Precisión macro	Recall macro	F1 macro
Random Forest	90.42	90.81	90.18	90.42
KNN	83.61	83.74	83.70	83.42
Decision Tree	75.32	75.60	74.22	74.66
SVM	72.56	73.26	72.54	72.33

Resultados de prueba

Después de fijar las configuraciones, cada modelo se entrenó nuevamente con las 2,534 muestras de entrenamiento y se evaluó sobre las mismas 543 muestras de prueba. No se combinaron entrenamiento y validación ni se ajustaron parámetros con test.

Modelo	Accuracy	Precisión macro	Recall macro	F1 macro
Random Forest	90.06	90.54	89.86	89.99
KNN	82.87	83.12	82.90	82.69
Decision Tree	74.03	74.51	74.02	74.04
SVM	70.72	70.96	70.70	70.59

Los valores proceden de los archivos validation_metrics.csv de cada modelo y results/final_test/test_metrics.csv. Se muestran dos decimales, pero la selección utilizó los valores completos

Matrices de confusión del conjunto de prueba

En las cuatro matrices, las filas representan las actividades reales y las columnas las predicciones. Los valores de la diagonal corresponden a aciertos; los valores fuera de ella indican confusiones.

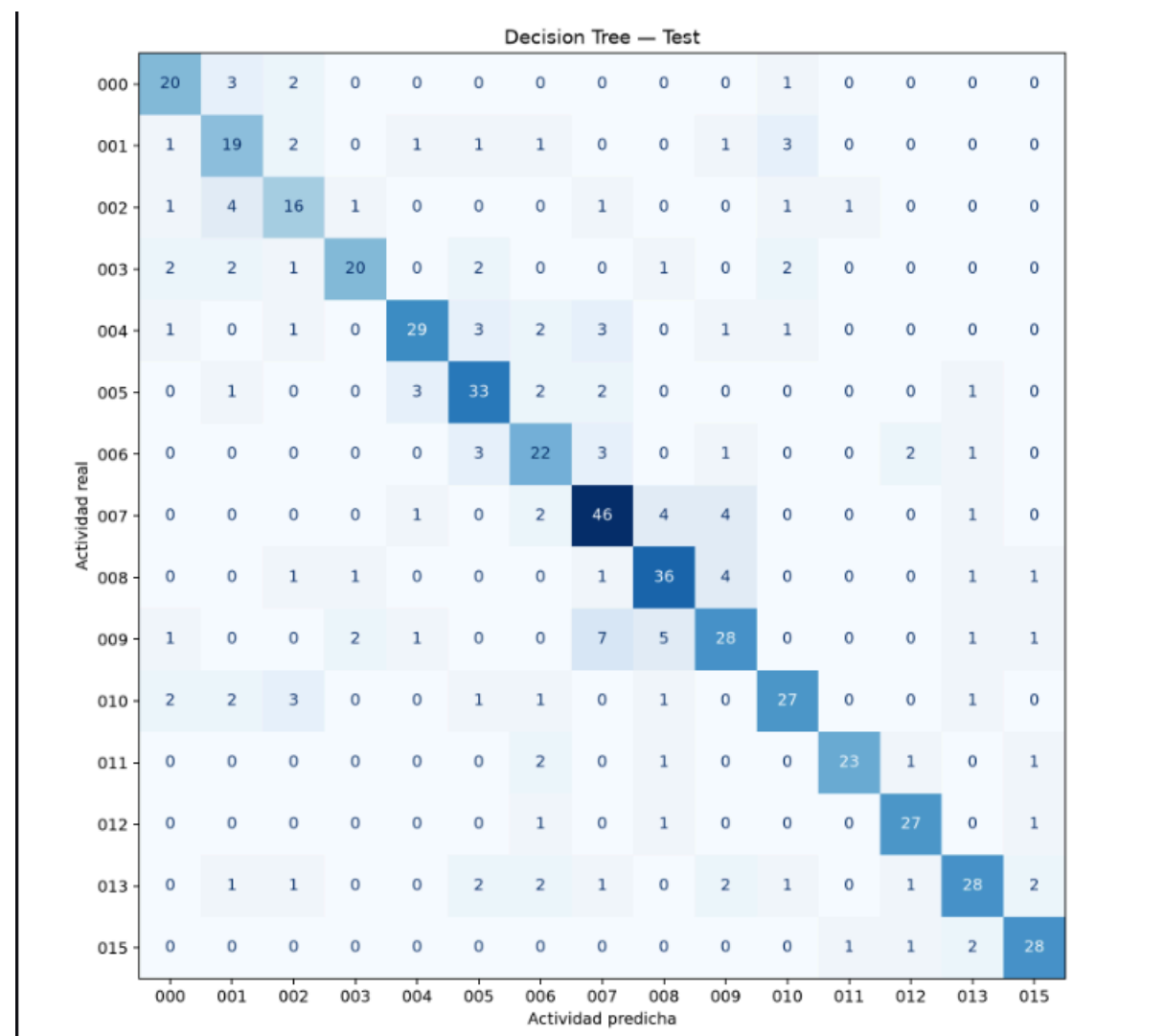


Figura 1. Matriz de confusión de Decision Tree sobre 543 muestras de prueba. El modelo clasificó correctamente 402 muestras. Entre sus errores, asignó la actividad 007 a siete muestras reales de 009, y la actividad 008 a otras cinco muestras de 009.

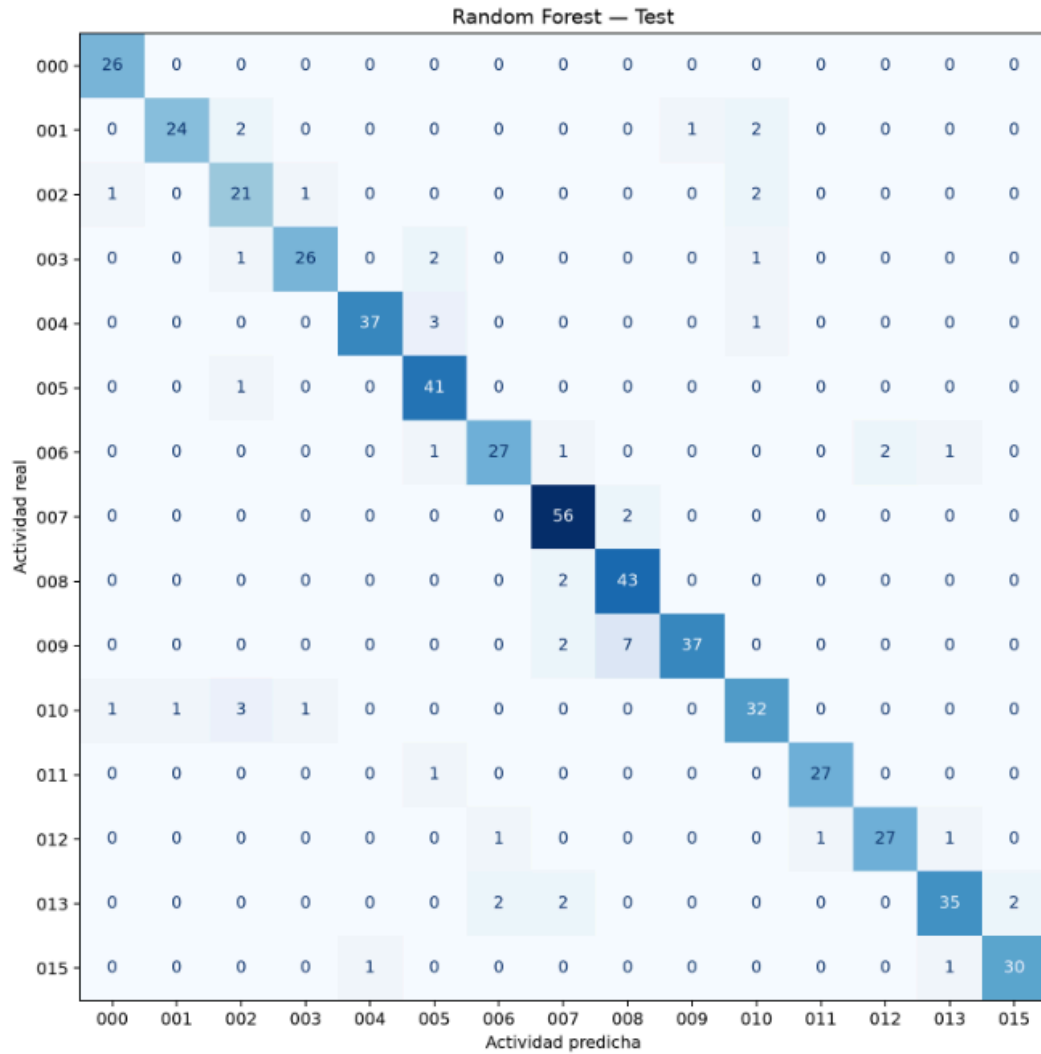


Figura 2. Matriz de confusión de Random Forest sobre 543 muestras de prueba. El modelo clasificó correctamente 489 muestras. Reconoció las 26 muestras reales de 000 y 41 de las 42 muestras de 005. Su mayor confusión individual fue predecir 008 para siete muestras reales de 009.

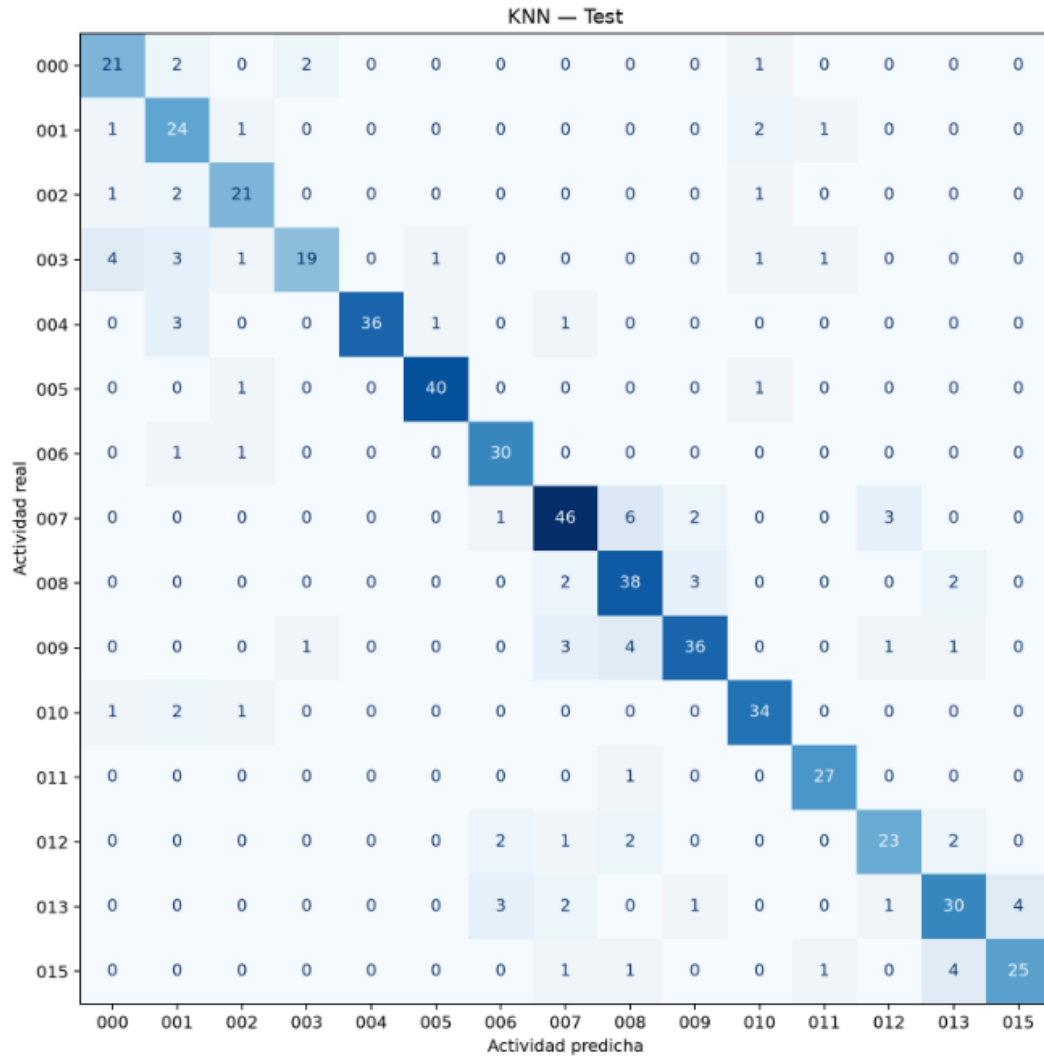


Figura 3. Matriz de confusión de KNN sobre 543 muestras de prueba. El modelo clasificó correctamente 450 muestras. Reconoció 30 de las 32 muestras de 006, pero confundió seis muestras reales de 007 con 008. También presentó confusiones entre 013 y 015, con cuatro errores en cada dirección.

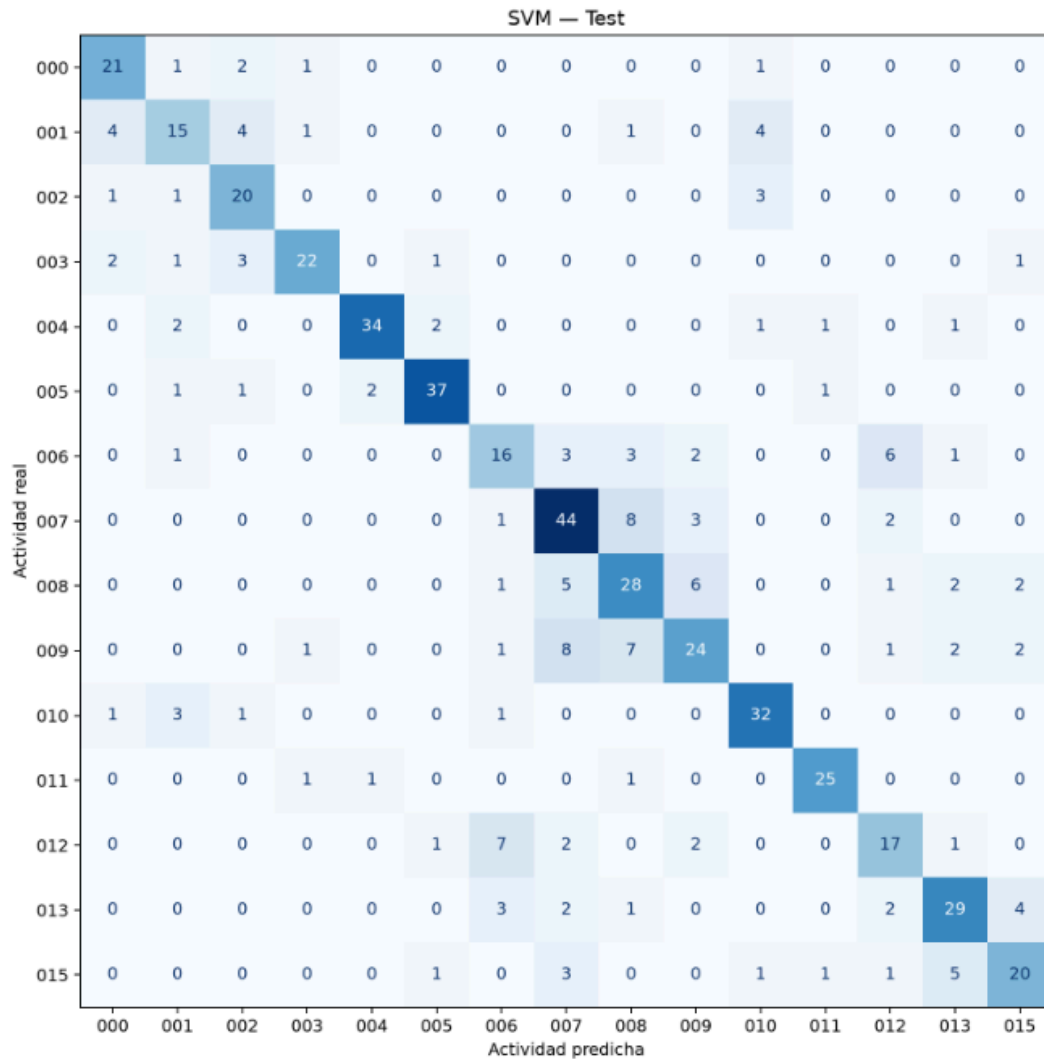


Figura 4. Matriz de confusión de SVM sobre 543 muestras de prueba. El modelo clasificó correctamente 384 muestras. Confundió ocho muestras de 007 con 008. Para la actividad real 009, predijo 007 en ocho casos y 008 en siete. También hubo confusión entre 006 y 012: seis muestras de 006 se predijeron como 012 y siete de 012 como 006.

Las matrices muestran que el resultado global no implica el mismo desempeño en todas las actividades. Por ejemplo, KNN reconoció más muestras de 006 que Random Forest, aunque Random Forest obtuvo las mejores métricas globales.

5. Explicación de las decisiones de selección y entrenamiento

Dataset y particiones comunes: Se conservó la versión compartida de 15 clases y 24 características para que todos los modelos resolvieran exactamente el mismo problema. Se utilizaron las particiones recibidas, correspondiente a 70/15/15. La preparación acordada establece estratificación y semilla 42, los scripts de entrenamiento no generaron nuevas divisiones.

Estandarización: Se estandarizaron las entradas de KNN y SVM para controlar el efecto de las distintas escalas. El escalador aprendió sus parámetros únicamente de entrenamiento para evitar incorporar información de validación o prueba. Los árboles trabajaron con las variables originales porque sus divisiones se basan en umbrales individuales.

Parámetros comparados: En Decision Tree se variaron profundidad y tamaño mínimo de hoja para comparar distintos niveles de complejidad. En Random Forest también se varió el número de árboles. En KNN se compararon tamaños de vecindario, pesos y distancias. En SVM se compararon kernels y valores de C y gamma. La comparación se limitó a las combinaciones indicadas, sin realizar una búsqueda exhaustiva.

Métrica principal: Se seleccionó F1 macro de validación porque calcula el F1 de cada actividad y asigna el mismo peso a las 15 clases. También se registraron accuracy, precision macro y recall macro. Se utilizó zero_division=0 para los casos de métricas indefinidas.

Empates: Cuando dos configuraciones obtuvieron exactamente el mismo F1 macro, se conservó la primera evaluada. Por ejemplo, KNN con un vecino y distancia Manhattan empató entre voto uniforme y ponderado por distancia, se conservó el uniforme.

Modelos	F1 macro de entrenamiento	F1 macro de validación
Decision Tree	100.00%	74.66%
Random Forest	100.00%	90.42%
KNN	100.00%	83.42%
SVM	91.58%	72.33%

Decision Tree y SVM presentaron diferencias amplias entre entrenamiento y validación, compatibles con sobreajuste. Random Forest también alcanzó un ajuste perfecto en entrenamiento, pero conservó un mejor desempeño en validación.

En KNN con un vecino, cada muestra de entrenamiento puede encontrarse a sí misma durante la predicción. Por ello, su F1 de entrenamiento de 100% no demuestra capacidad de generalización. El resultado del entrenamiento no se utilizó para seleccionar modelos.

Selección final. Random Forest superó a KNN, el segundo modelo, por aproximadamente 7.01 puntos porcentuales de F1 macro en validación: fue seleccionado con 100 árboles, profundidad sin límite explícito, una muestra mínima por hoja y `max_features="sqrt"`.

En prueba obtuvo 90.06% de accuracy y 89.99% de F1 macro. Su F1 disminuyó aproximadamente 0.44 puntos porcentuales respecto a validación, calculados con los valores sin redondear. No se modificaron parámetros ni se cambió el modelo seleccionado después de observar test.

La conclusión se limita a los modelos, configuraciones y partición de datos utilizados durante esta etapa del proyecto

6. Scripts y documentación en el repositorio

Los archivos utilizados para entrenar y evaluar los modelos son:

- `hector/decision_tree_model.py`
- `hector/random_forest_model.py`
- `elisheba/knn_model.py`
- `elisheba/svm_model.py`
- `evaluate_models.py`